Torneo de Komikan concurrente: informe de diseño

Tarea 2, Sistemas Operativos Benjamín Aguilera, Matías Marchant

1. Diseño del esquema de sincronización

El programa es un solo proceso con hilos POSIX, así que no usamos memoria compartida entre procesos: todos los hilos ven las mismas estructuras. El hilo principal hace de servidor y empareja a los jugadores. Cada jugador es un hilo (N en total) y cada mesa activa tiene 14 hilos: el de la mesa, que hace de juez, y los 13 de las fichas (1 puma y 12 perros). Aparte hay un hilo que solo espera señales. El tablero lo representamos como un grafo de 28 nodos, la cuadrícula de 5×5 más los 3 del triángulo, con una lista de vecinos y otra de saltos posibles para cada nodo. No incluimos diagonales en la cuadrícula.

Recurso	Primitiva	Para qué la usamos
Estado de una partida	`pthread_mutex_t` por mesa	Protege celdas, posiciones, contador de movimientos y la marca de término.
Retardo de las fichas y fin de partida	`pthread_cond_t` con reloj `CLOCK_MONOTONIC`	Cada ficha espera su retardo con `timedwait`; el juez espera ahí a que termine la partida.
Máximo de M mesas	`sem_t` iniciado en M	El servidor hace `sem_wait` antes de abrir una mesa y la mesa hace `sem_post` al cerrar.
Cola de llegada y contadores	Mutex del servidor, condiciones `llegada` y `apagar`	Cola FIFO de jugadores, partidas en juego, pico de mesas y abandonos.
Fin de cada jugador	Mutex y condición propios	El jugador queda bloqueado hasta que su mesa lo libera.
Log	Mutex global del registro	Cada línea (o tablero) se escribe entera, sin mezclarse con otras mesas.
SIGINT, SIGTERM	`pthread_sigmask` y `sigwait`	Apagado ordenado desde un hilo normal, no desde un manejador.

Movimientos sin turnos

Cada ficha repite lo mismo: espera un tiempo al azar entre `delay_min_ms` y `delay_max_ms`, revisa si la partida terminó, mira qué casillas tiene libres, elige una jugada al azar, la hace y evalúa si alguien ganó. La espera se hace con `pthread_cond_timedwait`, que suelta el mutex mientras duerme, así que las 13 fichas están compitiendo de verdad por el tablero y no hay un orden de turnos. Cuando el tiempo se cumple, la ficha vuelve con el mutex tomado y hace todo lo demás sin soltarlo. Por eso no puede pasar que una ficha vea una casilla libre y otra se la quite antes de moverse: si dos perros quieren la misma casilla, el que toma el mutex primero se mueve y el otro ya la encuentra ocupada y elige otra. Igual, `mover` y `capturar` vuelven a revisar que el destino sea vecino, esté vacío y que el salto exista. Cuando el puma captura, el perro queda marcado como muerto y su hilo termina en la siguiente vuelta.

Al principio las fichas esperaban con `sleep_for`. Funcionaba, pero al terminar la partida (o con Ctrl+C) cada hilo seguía durmiendo hasta el final de su retardo. Con la variable de condición, el `broadcast` que hace la partida al terminar las despierta a todas de inmediato.

Abandono y Ctrl+C

`prob_abandono` es el porcentaje de mesas en que un jugador se cae. En esas mesas se sortea al principio qué lado se cae y en qué movimiento (entre 1 y 40). Cuando llega ese movimiento, la primera ficha de ese lado que despierta llama a `pthread_exit` con el mutex de la partida tomado. Para que eso no deje colgada la mesa, cada ficha registra con `pthread_cleanup_push` una función que escribe el evento en el log, da la victoria al otro jugador, hace el `broadcast` y suelta el mutex. El juez despierta, cierra la partida, el servidor suma el abandono y la mesa hace `sem_post`. Como chequeo extra, al destruir la partida revisamos que `pthread_mutex_destroy` no devuelva error; en glibc devuelve `EBUSY` si el mutex sigue tomado, y nunca nos pasó.

Para Ctrl+C bloqueamos SIGINT y SIGTERM con `pthread_sigmask` antes de crear cualquier hilo, para que todos hereden la máscara y la señal no le llegue a una ficha cualquiera. Un hilo dedicado espera con `sigwait`. Como es un hilo normal y no un manejador asíncrono, puede tomar mutex y usar condiciones sin problema. Al recibir la señal marca el apagado, interrumpe las partidas que están en `en_juego`, hace `broadcast` para despertar al emparejador y a los jugadores que todavía no llegaban, y un `sem_post` por si el emparejador estaba bloqueado esperando mesa. Después el hilo principal hace `join` de todo y destruye mutex, condiciones y el semáforo. Al final el hilo de señales se cierra mandándole SIGUSR1, que está en el mismo conjunto.

2. Análisis de riesgos de concurrencia

Deadlock. Hay tres niveles de mutex y siempre se toman en el mismo orden: servidor, partida, registro. El hilo de señales toma el del servidor y adentro el de cada partida para interrumpirla; una ficha toma el de su partida y adentro el del log. No encontramos ningún camino que los tome al revés, así que no se puede formar un ciclo de

espera. Dentro de una mesa hay un solo mutex, entonces las fichas no se pueden bloquear entre ellas. El caso que más nos preocupaba era una ficha muerta con el mutex tomado, que dejaría pegados a los otros 12 hilos y al juez, y eso lo resuelve el manejador de limpieza.

Livelock. No hay reintentos activos. Si una ficha no tiene jugada, no insiste: vuelve a esperar un retardo completo, y como el retardo es aleatorio los hilos se van desfasando. Además el juego siempre avanza: después de cada movimiento se revisa si el puma quedó acorralado, así que mientras la partida sigue el puma tiene alguna jugada. Si nadie gana, `max_movimientos` corta la partida como empate.

Inanición. El mutex de pthreads no asegura orden FIFO, así que en teoría una ficha podría perder siempre. En la práctica hay muy poca contención (13 hilos, retardos de 50 a 300 ms y secciones críticas cortas) y el retardo aleatorio hace que ningún hilo tenga ventaja fija. Por el lado del servidor, los jugadores se atienden en orden de llegada y el único que espera en el semáforo es el emparejador, así que los pares no compiten entre sí por las mesas.

Otros riesgos. Todas las esperas en condiciones están dentro de un `while` que revisa el predicado, por los despertares espurios. Para no interrumpir una partida ya destruida, la mesa la saca de `en_juego` con el mutex del servidor tomado antes de destruirla. Las esperas con tiempo usan `CLOCK_MONOTONIC`, para que un cambio de hora del sistema no las alargue ni las corte.

3. Evaluación

Alternativas. Elegimos un mutex por mesa. Pensamos en un mutex por casilla, tomados en orden de índice para evitar deadlock, que en teoría deja mover a varias fichas a la vez. El problema es que un salto del puma toca tres casillas (origen, perro y destino), que las capturas y la condición de victoria son estado de toda la partida y necesitarían otro lock igual, y que habría que volver a validar la jugada después de tomar los locks. Un `rwlock` tampoco servía, porque cada intento de una ficha es leer y escribir de forma atómica, nunca solo leer.

Rendimiento. Con retardos uniformes entre 50 y 300 ms, el promedio es 175 ms, así que una mesa con 13 fichas hace como máximo unos 74 intentos por segundo. Si la sección crítica dura del orden de 50 μs, el mutex queda ocupado cerca de un 0,4 % del tiempo y casi nunca se encuentra tomado. O sea, lo que limita la velocidad son los retardos y no el lock, y entre mesas no hay espera porque cada una tiene su mutex. Lo medimos con ./torneo 10 5 config.txt: las 5 mesas jugaron a la vez y sumaron 763 movimientos. El torneo completo duró 4,0 s (incluye la llegada de los jugadores) y cada mesa jugó menos que eso, así que por mesa fueron entre 53 y 60 movimientos por segundo. Es menos que 74 porque los intentos en que la ficha no tiene casilla libre no cuentan como movimiento, y porque a medida que el puma captura quedan menos perros moviéndose.

Limitaciones. Un mutex por mesa es fácil de razonar, pero el log se escribe (con `flush`) dentro de la sección crítica de la partida, y su mutex es lo único que comparten todas las mesas; con `log_consola=1` esa escritura es más lenta y alarga la sección crítica. Los hilos de mesa recién se unen al final del torneo, aunque los de las fichas sí se unen al cerrar cada partida. La semilla hace repetibles los retardos y las jugadas elegidas, pero no el resultado completo, porque el orden en que las fichas toman el mutex depende del planificador: con la misma semilla una mesa terminó en 227 movimientos en una corrida y en 111 en otra. Y el abandono es simulado; con hilos, un fallo real tumbaría el proceso entero.

Pruebas. Corrimos Valgrind (memcheck) con abandono en todas las mesas (`./torneo 6 2 config_abandono.txt`) y terminó con 801 asignaciones, 801 liberaciones y cero errores. También probamos con ThreadSanitizer y AddressSanitizer sin avisos, y con Ctrl+C en distintos momentos del torneo: las partidas activas se cierran como interrumpidas y el programa sale normalmente.